**姓名：** **学号：** **班级：**

☯ 将你的**卷号**（在右上角或左下角）抄写到答卷本**封面**的右上角。

☯ 请在**答卷本**上作答，本**试卷**上的作答无效；**彩笔**、**铅笔**作答都将视作无效

☯ 若无特别说明，这里提及的概念、术语均以**讲义**为准，数据结构、算法等均为本课程讲授的**标准**版本

☯ 计算题除了给出**最终**的结果，还需列出计算**依据**及**过程**

## 1. 判断（请分别用 O、X 示意正、误） 2 x12 = 24

*01.*

*02.*

*03.*

*04.*

---

*05.*

*06.*

*07.*

*08.*

---

*09.*

*10.*

*11.*

*12.*

## 2. 填空（代数式须化简，常数保留两位小数） 4 x6 = 24

**A.** $T(n) = \sum_{k=1}^{2025} 64^k \cdot T(n/2^k) + \sqrt[2025]{\prod_{k=6}^{15} n^{k^3}} = \mathcal{O}($ )。

**B.** 采用二分查找算法（版本**C**），若**失败**情况的**平均查找长度**不小于**5**，则向量的长度至少是（ ）。

**C.** 用**交换版** List::**selectionSort**()对序列{**2025,...,3,2,1**}排序，有（ ）次交换是**原地**的。

**D.** 若通过**enqueue**()接口，将{**5,2,3,1,6,4,8,9,7,10**}依次插入一个初始为空的（压缩版）**最大队列**，则在此过程中，从队列**Y**中**dequeueRear**()的元素依次为（ ）。

**E.** 长度为**100**的某序列若共包含**4560**个逆序对，则其中**至少**有（ ）个完全落在区段[**10,80**)内。

**F.** 若字符频数为{ A:**2025**, B:**1013**, C:**507**, D:**254**, E:**x** }。
试写出所有**正整数**x，使得构造的**Huffman**树中存在权重**相等**的节点：（ ）。

## 3. Vector::expand() 6

若向量的**expand**()算法改为按$\{ 1^2, 2^2, 3^2, 4^2, \dots \}$来逐渐**扩容**，则在从**空向量**开始**连续**调用$n$次**insert**()的过程中，**expand**()**累计**消耗的时间为多少？为什么？



CST-2025F-123

CST-2025F-123

## 4. Expression Tree & RPN 5 + 5 = 10

考查仅包含**2025**个**二目**运算符的**中缀**表达式**S**，对应的**表达式树**、**逆波兰表达式**分别记作**ET(S)**、**RPN(S)**。

若在**ET(S)**中，每个节点$v$都满足$size(lc(v)) \geq size(rc(v)) - 2$，则在对**RPN(S)**的求值[p313]过程中，辅助栈的**规模opnd.size()最大**可能达到多少？试**解释**原因，并示意性地**画出**这样的一棵**ET(S)**。

## 5. Amortization 2 + 2 + 2 + 10 = 16

```
template <typename T> void puded( T* A, int& n ) {
   for ( int k = 0, i; k < n; ) {
     for ( i = 0; A[i] != A[k]; i++ ); //CMP操作
     if ( i != k ) {
        while ( ++i < n )
           A[i-1] = A[i]; //MOV操作
        n--;
     } else
        k++;
   } //for
} //puded
```

**A.** 以上算法的功能是什么？

**B.** 若**A = {2,0,2,5,1,1,0,9}**，在**puded<int>(A,8)**过程中，反白标注的两类操作分别会执行多少次？

**C.** 若**A = {3,0,2,4,0,1,8,4}**呢？

**D.** 实际上，两类操作的**总次数**仅取决于数组的**长度n**，而与其**内容**无关。试给出公式，并予证明。

## 6. Postorder + Levelorder ~ Preorder 4 + 4 + 8 + 4 = 20

考查**有根有序**的多叉树，若将根节点记作**r**，则其孩子依次记作为**{$r_1$, $r_2$, ..., $r_d$}**。

- 为得到多叉树的**层次**遍历序列**level(r)**，只需对二叉树的**层次**遍历算法略加修改：

  任一节点出队并接受访问之后，该节点**所有**的孩子依次入队

- 仿照二叉树，还可分别递归地定义多叉树的**先序**、**后序**遍历序列为：

  **pre(r) = {r} + pre($r_1$) + pre($r_2$) + ... + pre($r_d$)** //“+”表示序列的**连接**

  **post(r) = post($r_1$) + post($r_2$) + ... + post($r_d$) + {r}**

我们进一步约定，树中的节点用**[0,n)**内的整数随机地**唯一**编号。

**A.** 若**post(r)={6,4,2,8,1,3,9,0,5,7}**、**level(r)={7,2,0,5,6,4,9,8,1,3}**，试**画出**多叉树的结构。

试实现如下算法，在$\mathcal{O}(n)$**时间**内、使用$\mathcal{O}(n)$辅助**空间**，由**post(r)**和**level(r)**导出对应的**pre(r)**。

```
PL2P( Vector<int>& post, Vector<int>& level, Vector<int>& pre );
```

**B.** 介绍你的设计**思路**及背后的**原理**。

**C.** 用伪代码**描述**你的算法。

**D.** **证明**时间、空间复杂度均符合要求。

CST-2025F-123

## Akra-Bazzi Theorem

68

**分治策略对应的递推式，更一般地形如：** //k组：各含$a_i$个规模为$b_i n+h_i(n)$的子任务

$T(n) = \sum_{i=1}^{k} a_i \cdot T(b_i \cdot n + h_i(n)) + \mathcal{O}(g(n))$ //任务的划分、解的合并，总共耗时g(n)

- $a_i > 0$，$0 < b_i < 1$，$|h_i(n)| = \mathcal{O}(n / \log^2 n)$
- $g(n) \geq 0$，**且有常数** $d > 0$**使得** $|g'(n)| = \mathcal{O}(n^d)$ //多项式增长

❖ **只要取** $p$ **使得** $\sum_{i=1}^{k} a_i \cdot b_i^p = 1$，**则** $T(n) = \Theta(n^p \cdot (1 + \int_1^n g(u)/u^{p+1} du))$ //与$h_i(n)$无关

✖ **坏选择：** $T(n) = 1 \cdot T(3/4 \cdot n) + 1 \cdot T(1/4 \cdot n) + \mathcal{O}(n) = \mathcal{O}(n \log n)$ //p=1 |linearSelect

✓ **好选择：** $T(n) = 1 \cdot T(3/4 \cdot n) + 1 \cdot T(1/5 \cdot n) + \mathcal{O}(n) = \mathcal{O}(n)$ //p<1 |算法（第14章）

❖ $T(n) = 2025^2 \cdot \sum_{k=1}^{9} T(k/2025 \cdot n) + \sqrt[2025]{\prod_{k=3}^{12} n^{k^3}} = \mathcal{O}(n^3 \cdot \log n)$



## 问题 + 蛮力算法

70

❖ **从整数序列中，找出总和最大的区间（有多个时，短者、靠后者优先）**

$\mathcal{A}[0,19) = \{ 1, -2, 7, 2, 6, -9, 5, 6, -12, -8, 13, 0, -3, 1, -2, 8, 0, -5, 3 \}$

```
int gs_BF( int A[], int n ) //蛮力策略：O(n^3)
   int gs = 0; //目前已知的最大和：零宽度区间，总和为零
   for ( int lo = 0; lo < n; lo++ ) //枚举所有的
      for ( int hi = lo+1; hi <= n; hi++ ) //O(n^2)个区间!
         int s = sum( A, lo, hi ); //用O(n)时间求和
         gs = max( gs, s ); //择优更新
   return gs;
```






## 实现

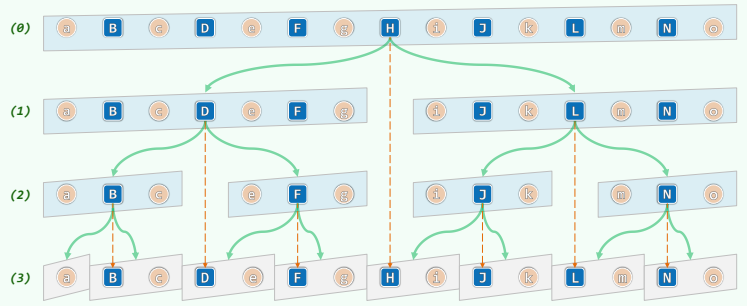
146

```
template <typename T> Rank binSearch( T* S, const T& e, Rank lo, Rank hi ) {
   while ( lo < hi ) { //不变性：[0, lo) <= e < [hi, n)
      Rank mi = (lo + hi) / 2;
      if ( e < S[mi] ) hi = mi; //[lo, mi)
      else lo = mi + 1; //(mi, hi)
   } //出口时，[lo = hi] = M
   return lo - 1; //至此，[lo]为大于e的最小者，故[lo-1] = m即为不大于e的最大者
}
```



## 平均查找长度：$ASL_{succ} = ASL_{fail} = \mathcal{O}(1.00 \cdot \log n)$

149






## 基本版：用模板封装一下

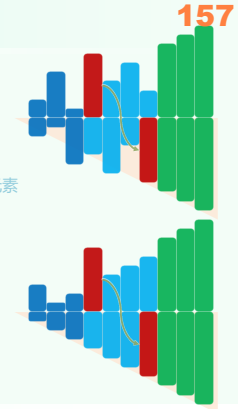
157

❖ 

```
template <typename T>
void Vector<T>::bubbleSort( Rank lo, Rank hi ) {
   while ( lo < --hi ) //逐趟起泡扫描
      for ( Rank i = lo; i < hi; i++ ) //逐对检查相邻元素
         if ( _elem[i] > _elem[i + 1] ) //若逆序
            swap( _elem[i], _elem[i + 1] ); //则交换
}
```

❖ $\Theta(n^2)$：**不识好坏，千篇一律**

❖ **然而，不同输入的混乱程度毕竟有别，比如...**



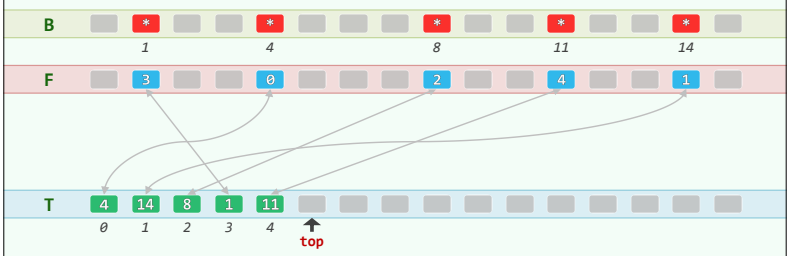


## J. Hopcroft, 1974: Rank F[m], T[m], top = 0;

182

❖ **为区分三种状态，需要记录更多信息：拆分数组 + 校验环：**

To[From[k]] == k & From[To[i]] == i //君右杜左，虎符会合





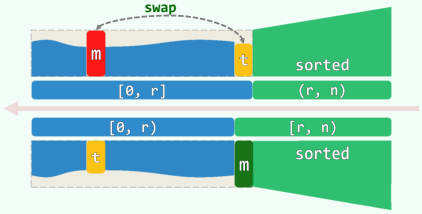

## 采用交换法的选择排序

232

❖ **交换之前：m与t同属一个轮换C，且前后紧邻**

❖ **交换之后：**

- **m从C中逸出，并自成一个轮换**
- **C的长度减一（可能归零）**
- **其余的轮换，保持不变**




❖ **C含k个轮换，就会出现k次多余的交换**

- $1 \leq k \leq n$

❖ **多余的交换 ~ C仅含m ~ m已经就位 ~ t在前缀中最大**

- $\mathbb{E}(k) = 1/1 + 1/2 + 1/3 + \cdots + 1/n = \Theta(\log n)$



## 实现

236

```
template <typename T> //逐一引入节点r，持续拓展有序前缀(h,r)
void List<T>::insertionSort( LNP<T> h, LNP<T> t ) {
   for ( LNP<T> r = h->succ, s = r->succ; r != t; r = s, s = r->succ )
      r->moveSucc( search( h, r->data, r ) ); //为[r]在(h,r)内找到适当位置并插入
} //仅使用O(1)辅助空间，属于就地算法
```

❖ **得益于此前约定的search()接口语义，前缀的确总是保持有序，而且稳定**

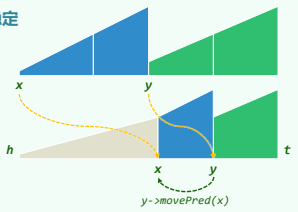
❖ **验证以下情况，体会哨兵的作用：前缀中有元素与r相等；r在前缀中最小/最大；前缀为空；...**



## 二路归并

241

```
template <typename T> LNP<T> //将[y,t)中的节点，分别转移至(h,y)中的适当位置
List<T>::merge( LNP<T> h, LNP<T> y, LNP<T> t ) {
   for ( LNP<T> x = h->succ; (x != y) && (y != t);  )
      if ( x->data > y->data ) //小者优先，保证稳定
         (y = y->succ)->pred->movePred( x );
      else
         x = x->succ;
} //O(n)，线性正比于节点总数
```






## 主算法

303

```
double evaluate( char* S, char* RPN ) { //S保证语法正确
   Stack<double> opnd; Stack<char> optr; //运算数栈、运算符栈
   for ( optr.push('\0'); !optr.empty();  ) //逐个处理各字符，直至运算符栈空
      if ( isdigit( *S ) ) //若为运算数（可能多位、小数），则
         readNumber( S, opnd ); //读入
      else //若为运算符，则视其与栈顶运算符之间优先级的高低
         switch( prio( optr.top(), *S ) ) { /* 分别处理 */ }
   return opnd.pop(); //弹出并返回最后的计算结果
} //evaluate
```

## 快捷求值 313

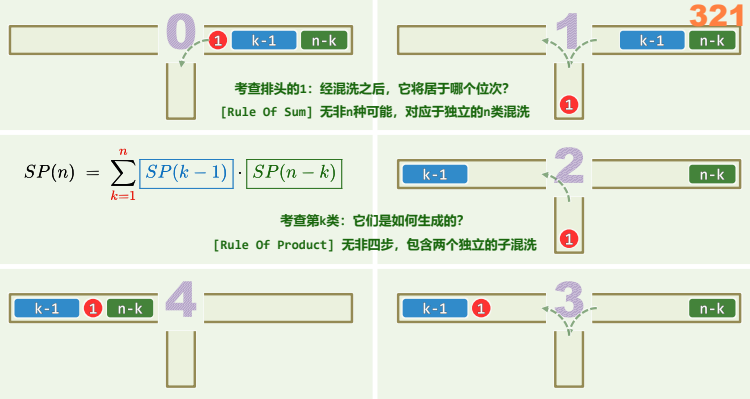
0 ! 123 + 4 5 6 ! * 7 ! 8 / + * 9 / +

630
4230
1880
2004

```
double rpn( char* S ) { //对RPN求值
   Stack<double> opnd; //仅需一个运算数栈
   for ( char* item = strtok( S, " " ); item; item = strtok( NULL, " ") )
      if ( 1 < strlen( item ) || isdigit( item[0] ) ) //遇到操作数
         opnd.push( atof( item ) ); //则令其入栈
      else //遇到运算符
         opnd.push( calcu( opnd, item[0] ) ); //则直接计算
   return opnd.pop(); //唯一尚存的操作数，即为计算结果
} //rpn
```

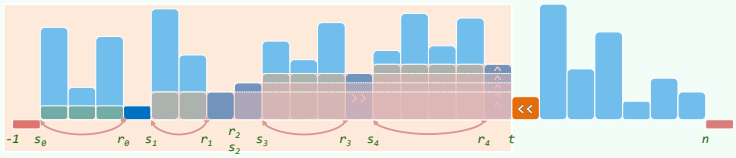


## 321






## One-Pass Scan: Algorithm 347



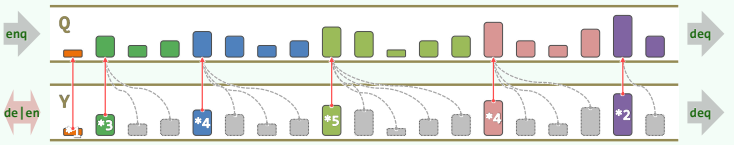
```
Stack<Rank> SR;  uint64_t maxRect = 0; //SR.2ndTop() == s(r)-1 & SR.top() == r
for ( Rank t = 0; t <= n; t++ ) //amortized-O(n)
   while ( !SR.empty() && ( t == n || H[SR.top()] >= H[t] ) )
      Rank r = SR.pop(), s = SR.empty() ? 0 : SR.top() + 1;
      maxRect = max( maxRect, H[r] * ( t - s ) );
   if ( t < n ) SR.push( t );
return maxRect;
```



## Max Queue：均摊成本 354

❖ enqueue(e) //均摊$O(1)$

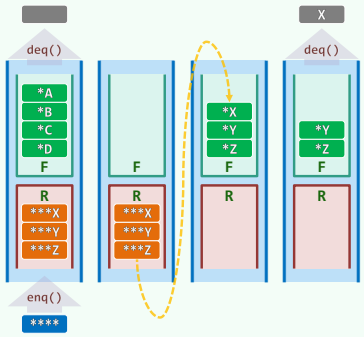
```
Q.enqueue(e); c = 1;
while ( !Y.empty() && (e => Y.rear()->data) )  c += Y.dequeueRear()->count;
Y.enqueue( <&e, c> );
```




❖ dequeue() { if ( --Y.top().count < 1 ) Y.dequeue(); return Q.dequeue(); } //$O(1)$



## Accounting|记账法|归因法 357




❖ Assign 4 coins to each element
//deposit | pre-paid
- 1 for its enqueue()
- 2 for transfer, and
- the last 1 for dequeue()

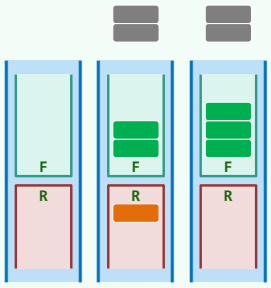
❖ Hence every operation is pre-paid and ...

❖ The structure will never run out of credit

❖ Amortized cost of any operation sequence involving n ITEMs is
4n = $O(n)$



## Aggregate|聚合法 358




❖ Consider the moment when
d dequeue()'s and e enqueue()'s have been done //d <= e

❖ The time cost for ALL the operations is

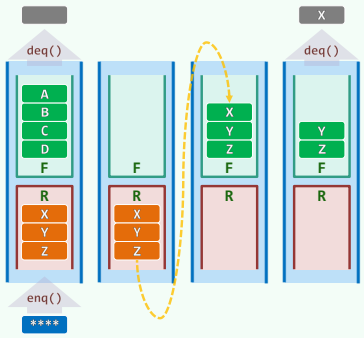
$$\leq 4 \cdot d + 3 \cdot (e - d) = 3e + d$$

❖ The amortized cost for each OPERATION is

$$\frac{3e+d}{e+d} < 3 = \mathcal{O}(1)$$



## Potential|势能法 359




❖ Consider the $k^{th}$ operation

❖ Define $\Phi_k = |R_k| - |F_k|$

❖ Then $A_k = T_k + \Phi_k - \Phi_{k-1} \equiv 2$

❖ Hence

$$2n \equiv \sum_{k=1}^{n} A_k = \sum_{k=1}^{n} T_k + \Phi_n - \Phi_0$$

$$2n = T(n) + \Phi_n - \Phi_0 > T(n) - n$$

$$T(n) < 3n = \mathcal{O}(n)$$



## 有根有序树 362

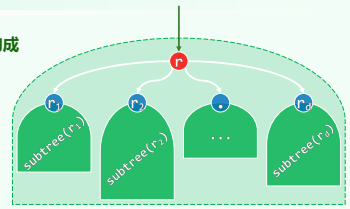
❖ 树也是图，由顶点（vertex）和联边（edge）构成
顶点对应的数据结构为节点，不妨混称

❖ 指定某一节点为根后，即是有根树
tree rooted at r = tree(r)

❖ 任给若干棵有根子树（subtree）$\{r_1, r_2, \ldots, r_d\}$
引入新节点 $r$，并向 $r_i$ 分别发出一条有向边，便可得到一棵（更大、更高的）有根树

❖ $r$ 和 $r_i$ 互称父（parent）、子（child）；边数 $d$ 为 $r$ 的度（degree）
零度节点称叶子（leaf），其余为内部节点（internal node）

❖ 兄弟（sibling）之间若依 $\{r_1, r_2, \ldots, r_d\}$ 排序，则是有序树（ordered tree）



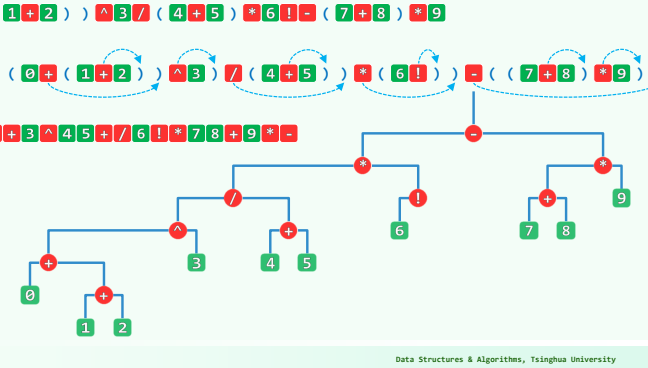



## 后序遍历实例：表达式树：后序遍历 = RPN 391

( 0 + ( 1 + 2 ) ) ^ 3 / ( 4 + 5 ) * 6 ! - ( 7 + 8 ) * 9

( ( ( ( ( 0 + ( 1 + 2 ) ) ^ 3 ) / ( 4 + 5 ) ) * ( 6 ! ) ) - ( ( 7 + 8 ) * 9 ) )

0 1 2 + + 3 ^ 4 5 + / 6 ! * 7 8 + 9 * -



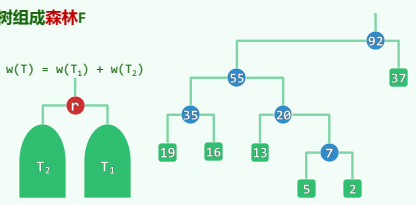



## Huffman算法，1952 434

❖ 贪心策略：自底而上地逐个拼接：频数小|大的字符，更早|晚拼接

❖ 字符各自成一棵树，权重即其频数；n棵树组成森林F

```
while ( F中的树不止一棵 )
   取出频率最小的两棵树：T1和T2
   将它们合并成一棵新树T
      - 以T1、T2为左、右子树
      - 权重取作T1与T2权重之和
最终唯一剩下的那棵树，便是一棵OWCT
```




❖ 贪心策略通常并不能保证得到最优解，Huffman的算法是个例外吗？